UNIVERSIDAD TECNOLÓGICA DE LA RIVIERA MAYA

INGENIERIA EN DESARROLLO Y GESTION DE SOFTWARE

GESTIÓN DEL PROCESO DE DESARROLLO DE SOFTWARE.

TSU. SEANI ARCOS GOMEZ - 2202011

TSU. CARLOS ANTONIO BALCAZAR OLAN - 2202035

TSU. YACIEL AARON CAMARA CHAN - 2202030

TSU. GEOVANI DE JESUS CARRILLO MORALES - 2202004

TSU. JUAN DIEGO LOPEZ MIS - 2202250

TSU. GUADALUPE MONSERRAT NUÑEZ NANGULLASMU - 2102417

ING. JUAN ANTONIO DOMINGUEZ JUAREZ

08 DE MARZO DE 2026

Contenido

Planeación Del Proceso De Desarrollo De Software 4

1. Descripción Del Proyecto Integrador 4

1.1. Información Básica 4

1.2. Alcance 5

2. Parámetros De Configuración De Herramientas 7

2.1. Entorno De Desarrollo 7

2.2. CI/CD Pipeline 9

3. Plan De Pruebas 12

3.1. Objetivo Del Plan De Pruebas 12

3.2. Alcance De Las Pruebas 13

3.3. Tipos de Pruebas 15

3.4. Criterios De Aceptación 21

3.5. Ambiente De Pruebas 22

3.6. Aprobaciones Y Responsables 23

4. Casos De Pruebas 23

4.1. Formato De Caso De Pruebas 23

4.2. Ejemplo de Caso de Pruebas 34

4.3. Plantilla de Casos de Prueba 35

5. Flujo De Trabajo Para Control De Versiones 40

5.1. Estrategia de Branching Seleccionada 40

5.2. Estructura De Ramas 41

5.3 Nomenclatura De Ramas 43

5.4. Convención De Commits 44

5.5. Proceso De Pull Request 44

5.6. Proceso De Pull Request 45

5.7 Reglas De Protección De Ramas (Branch Protection Rules) 47

5.8 Flujo De Trabajo Diario 48

6. Estrategia De Despliegue 50

6.1 Estrategia Seleccionada 50
6.2. Ambientes De Despliegue 51
6.3. Proceso De Despliegue Detallado 52
6.4. Configuración Del Pipeline De Despliegue 52
6.5. Plan De Rollback 53
6.6. Métricas de Monitoreo 54
7. Repositorio Configurado 55
7.1 Estructura Del Repositorio 55
7.2 README.Md Completo 58
7.3 .gitignore Configurado 62
7.4 Configuración De Github 63
7.5 Issues Y Project Board 64
Bibliografía 67

Ilustración 1. Ramas 42
Ilustración 2. Branch protection rules 48
Ilustración 3. Proyecto Integrador - Sprint 1 65
Ilustración 4. Gestión De Issues 66

Planeación Del Proceso De Desarrollo De Software

1. Descripción Del Proyecto Integrador

1.1. Información Básica

Nombre Del Proyecto

Cobrixs

Objetivo General

Desarrollar una plataforma web moderna, eficiente y accesible que permita a los pequeños comercios gestionar de manera centralizada sus ventas, inventario y pagos a proveedores, garantizando actualización en tiempo real, control de usuarios y generación de reportes automatizados para apoyar la toma de decisiones.

Descripción Breve

CobriXS es una plataforma web diseñada para optimizar la gestión de pequeños comercios en México, especialmente minisúpers y tiendas de abarrotes. El sistema integra en un solo lugar el control de ventas, inventario en tiempo real y administración de pagos a proveedores, facilitando una operación más ordenada, rápida y confiable.

La plataforma utiliza Supabase como backend en la nube, permitiendo autenticación segura, almacenamiento en base de datos PostgreSQL y sincronización en tiempo real entre ventas e inventario. Incluye sistema de roles (administrador y empleado), generación de reportes en PDF y CSV, y control automatizado de movimientos financieros.

CobriXS se puede utilizar desde una computadora y cuenta con funciones accesibles desde dispositivos móviles, lo que brinda flexibilidad al negocio. También incorpora

opciones de accesibilidad, como ajustes de idioma y tipografía, para mejorar la experiencia del usuario.

El modelo de negocio se basa en suscripciones, ofreciendo un primer mes gratuito y posteriormente planes de 6 y 12 meses, adaptándose a las necesidades de cada comercio.

Stack Tecnológico Principal (Lenguajes, Frameworks, Bases De Datos*)*

- Frontend: HTML, CSS y JavaScript con React.
- Backend: Node.js.
- Base de datos: Supabase (PostgreSQL en la nube)
- Autenticación: Supabase Auth
- Arquitectura: Cliente–Servidor con backend como servicio (BaaS)

Tipo De Aplicación

Aplicación web (responsive), accesible desde navegador en computadora y con módulos operativos adaptados para dispositivos móviles.

1.2. Alcance

Módulos o features principales.

1. **Módulos de Inicio (Dashboard):** Visualización general del negocio: resumen de ventas del día, productos vendidos, stock disponible, alertas de bajo inventario y accesos rápidos a funciones clave.
2. **Módulos de Usuarios:** Creación, activación/inactivación y búsqueda de usuarios dentro del sistema.

3. **Módulos de Productos:** Registro de nuevos productos, consulta del catálogo, búsqueda y exportación en Excel.
4. **Módulos de Inventario:** Control de existencias, registro de movimientos, visualización de stock total y alertas de inventario bajo.
5. **Módulos de Panel de Cobro:** Apertura de caja, agregado de productos, consulta de precios y procesamiento de pagos con generación de ticket y corte de caja.
6. **Módulos de Pagos a Proveedores:** Registro de proveedores y control de pagos realizados.
7. **Módulos de Reportes:** Consulta y exportación de reportes en Excel y PDF, incluyendo ventas totales, transacciones, productos vendidos y tendencias básicas.
8. **Módulos de Configuración:** Opciones de apariencia, accesibilidad, idioma y ajustes generales del sistema.

Usuarios o roles del sistema.

- **Administrador:** Control total del sistema, gestión de usuarios, proveedores, inventario y reportes.
- **Empleado:** Operación de ventas, gestión de proveedores, consulta de productos, inventario y reportes básicos.

Casos de uso principales.

- **Control de Usuarios y Permisos:** Funcionalidad centralizada bajo la responsabilidad del Administrador, quien puede crear, modificar, activar o

desactivar usuarios, así como asignar roles y permisos de manera segura, garantizando un control adecuado del acceso al sistema.

- **Venta de Productos y Emisión de Ticket:** Proceso optimizado para el Cajero que permite agregar o escanear productos, calcular automáticamente el total de la venta, procesar distintas formas de pago y emitir el ticket correspondiente. Cada transacción actualiza el inventario en tiempo real y se integra al sistema para su control en el corte de caja.
- **Gestión de Inventario:** Administrado directamente por el sistema, permite la consulta de stock en tiempo real, el registro automático de movimientos tras cada venta, la generación de alertas por bajo inventario y un mejor control general de existencias.
- **Registro de Pagos a Proveedores:** Permite llevar un control unificado de los pagos realizados a proveedores, asegurando que el sistema actualice automáticamente el fondo de caja después de cada transacción, manteniendo así la precisión financiera.

2. Parámetros De Configuración De Herramientas

2.1. Entorno De Desarrollo

2.1.1. Control de versiones

- Plataforma: GitHub
- Estrategia de branching: GitHub Flow
- Convención de commits: Conventional Commits

[user]

```
        name = RED6908
        email = 2202035@utrivieramaya.edu.mx
[filter "lfs"]
        clean = git-lfs clean -- %f
        smudge = git-lfs smudge -- %f
        process = git-lfs filter-process
        required = true
[core]
        editor = code --wait
        autocrlf = input
[alias]
        st = status
        co = checkout
        br = branch
        ci = commit
        lg = log --oneline --graph --decorate
```

2.1.2. Contenedores (Docker)

Proporciona tu archivo Dockerfile y docker-compose.yml:

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 4000
ENV NODE_ENV=development
CMD ["npm", "run", "dev"]
```

Docker-compose.yml:

```
version: '3.8'
services:
  backend:
    build: .
    ports:
      - "4000:4000"
    environment:
      - SUPABASE_URL=${SUPABASE_URL}
      - SUPABASE_ANON_KEY=${SUPABASE_ANON_KEY}
      - NODE_ENV=development
    volumes:
      - .:/app
      - /app/node_modules
```

2.1.3. Variables De Entorno

Variables de entorno (sin incluir valores sensibles)

Archivo .env

```
# Supabase
VITE_SUPABASE_URL=https://cjqlzjhjtgpmlalzjtfx.supabase.co
VITE_SUPABASE_ANON_KEY=clave_privada
```

2.2. CI/CD Pipeline

2.2.1. Configuración de Github Actions

Archivo de workflow (.github/workflows/ci-cd.yml):

```
name: CobriXS CI/CD
```

```yaml
on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Setup Node.js
        uses: actions/setup-node@v3
        with:
          node-version: '18'
          cache: 'npm'
      - name: Install dependencies
        run: npm ci
      - name: Run linter
        run: npm run lint
      - name: Run tests
        run: npm test
  build:
    needs: test
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'
    steps:
```

```
    - uses: actions/checkout@v3
    - name: Build Docker image
      run: docker build -t cobrixs-v_2:latest .
    - name: Login to Docker Hub
      uses: docker/login-action@v2
      with:
        username: ${{ secrets.DOCKER_USERNAME }}
        password: ${{ secrets.DOCKER_PASSWORD }}
    - name: Tag Docker image
      run: docker tag cobrixs-v_2:latest ${{ secrets.DOCKER_USERNAME }}/cobrixs-
v_2:latest
    - name: Push Docker image
      run: docker push ${{ secrets.DOCKER_USERNAME }}/cobrixs-v_2:latest
```

2.2.2. Herramientas De Calidad De Código

- Linter: ESLint
- Formatter: Prettier
- Análisis estático: CodeClimate

Configuración ESLint (.eslintrc.json):

```
{
  "env": {
    "browser": true,
    "es2021": true,
    "node": true
  },
  "extends": [
```

```
    "eslint:recommended",
    "plugin:react/recommended"
  ],
  "rules": {
    "indent": ["error", 2],
    "quotes": ["error", "single"],
    "semi": ["error", "always"]
  }
}
```

2.2.3. Infraestructura Como Código (Iac)

El proyecto CobriXS no utiliza Infraestructura como Código, ya que el despliegue del sistema se realiza mediante plataformas de despliegue gestionadas y pipelines de integración continua.

La configuración y despliegue del sistema se documenta en las secciones de estrategia de despliegue y CI/CD, donde se describe el uso de GitHub, GitHub Actions y variables de entorno para automatizar el proceso de construcción y publicación del sistema.

3. Plan De Pruebas

3.1. Objetivo Del Plan De Pruebas

El objetivo del plan de pruebas es establecer un proceso sistemático para evaluar el correcto funcionamiento del sistema CobriXS y asegurar que cumpla con los requisitos establecidos durante el desarrollo del proyecto.

Mediante la ejecución de diferentes tipos de pruebas se busca garantizar que cada módulo funcione de manera adecuada, detectar errores antes de que el sistema sea liberado

a producción, validar los requisitos funcionales y no funcionales definidos en el proyecto y asegurar la calidad general del código desarrollado por el equipo.

Además, el plan de pruebas permite verificar que los procesos principales del sistema, como la gestión de ventas, el control de inventario, la administración de usuarios y el registro de pagos a proveedores, operen de manera integrada y mantengan la consistencia de los datos almacenados en la base de datos administrada mediante Supabase.

3.2. Alcance De Las Pruebas

El alcance del plan de pruebas del sistema CobriXS contempla la verificación de las funcionalidades principales de la plataforma, así como la validación de los flujos críticos que realizan los usuarios dentro del sistema. Estas pruebas permitirán asegurar que los módulos principales operen correctamente, que la información se procese de manera adecuada y que la interacción entre el frontend, el backend y la base de datos funcione sin errores.

Dentro Del Alcance

Las pruebas incluirán la validación de las siguientes funcionalidades principales del sistema:

- Módulos de Inicio (Dashboard): visualización general del estado del negocio dentro del sistema, incluyendo el resumen de ventas del día, productos vendidos, stock disponible, alertas de inventario bajo y accesos rápidos a las funciones principales.

- Gestión de usuarios: creación, edición, activación o desactivación de usuarios y asignación de roles dentro del sistema.
- Panel de ventas: registro de ventas, cálculo automático de totales, selección de productos y generación de tickets.
- Gestión de productos: registro de nuevos productos, consulta del catálogo, búsqueda de productos y actualización de información.
- Control de inventario: actualización automática del inventario después de cada venta, registro de movimientos de stock y alertas de inventario bajo.
- Gestión de proveedores: registro de proveedores y control de pagos realizados.
- Reportes del sistema: generación y consulta de reportes de ventas, inventario y movimientos financieros.
- Configuración del sistema: ajustes generales como idioma, apariencia y parámetros del sistema.

También se evaluarán los flujos críticos del usuario, entre los que destacan:

- Inicio de sesión y autenticación de usuarios.
- Registro completo de una venta desde la selección de productos hasta la generación del ticket.
- Actualización automática del inventario tras realizar una venta.
- Consulta de productos y existencias dentro del sistema.
- Registro de pagos a proveedores.

Asimismo, se probarán las integraciones entre los distintos componentes del sistema, verificando la correcta comunicación entre el frontend desarrollado en React, el

backend del sistema y la base de datos gestionada mediante Supabase, asegurando que los datos se almacenen, recuperen y actualicen correctamente.

Finalmente, se verificarán las validaciones de datos, incluyendo la verificación de formularios, autenticación de usuarios, restricciones de campos obligatorios, control de tipos de datos y consistencia de la información almacenada en la base de datos.

Fuera Del Alcance

En esta fase del proyecto no se contemplan pruebas avanzadas de rendimiento o seguridad. Por lo tanto, quedan fuera del alcance del presente plan de pruebas:

- **Pruebas de carga extensivas**, destinadas a evaluar el comportamiento del sistema bajo grandes volúmenes de usuarios simultáneos. Estas pruebas podrán realizarse en fases posteriores del proyecto.
- **Pruebas de penetración**, orientadas a identificar vulnerabilidades profundas de seguridad en el sistema.
- **Compatibilidad con navegadores legacy o versiones muy antiguas**, ya que CobriXS está diseñado para operar en navegadores modernos como Chrome, Edge, Firefox y Safari.

3.3. Tipos de Pruebas

Para garantizar la calidad y el correcto funcionamiento del sistema CobriXS se implementarán distintos tipos de pruebas durante el desarrollo del proyecto. Estas pruebas permitirán verificar tanto el funcionamiento individual de los componentes como la interacción entre los diferentes módulos del sistema.

3.3.1. Pruebas Unitarias

Las pruebas unitarias se utilizan para verificar el funcionamiento correcto de componentes individuales del sistema, como funciones, servicios o módulos específicos. Estas pruebas permiten asegurar que cada parte del código funcione correctamente de manera aislada antes de integrarse con otros componentes del sistema.

En el sistema CobriXS, las pruebas unitarias se aplicarán principalmente a funciones relacionadas con:

- Validación de datos en formularios.
- Procesamiento de ventas.
- Cálculo automático de totales en el panel de cobro.
- Actualización del inventario después de cada venta.
- Funciones de autenticación de usuarios.

- **Framework:** Jest
- **Cobertura objetivo:** 70–80% del código
- **Responsable:** Cada desarrollador prueba su propio código antes de integrarlo al repositorio principal.

Ejemplo de prueba unitaria (Jest):

```
// auth.test.js
describe('Authentication Service', () => {
 test('should hash password correctly', async () => {
   const password = 'myPassword123';
   const hashed = await hashPassword(password);
```

```
  expect(hashed).not.toBe(password);
  expect(hashed.length).toBeGreaterThan(20);
 });
 test('should validate correct password', async () => {
  const password = 'myPassword123';
  const hashed = await hashPassword(password);

  const isValid = await validatePassword(password, hashed);
  expect(isValid).toBe(true);
 });
 test('should reject incorrect password', async () => {
  const password = 'myPassword123';
  const hashed = await hashPassword(password);
  const isValid = await validatePassword('wrongPassword', hashed);
  expect(isValid).toBe(false);
 });
});
```

3.3.2. Pruebas De Integración

Las pruebas de integración tienen como objetivo verificar que los diferentes componentes del sistema funcionen correctamente cuando interactúan entre sí. En el caso de CobriXS, estas pruebas permitirán validar la comunicación entre el frontend, el backend y la base de datos.

Entre los procesos que se evaluarán mediante pruebas de integración se encuentran:

- Registro y consulta de usuarios dentro del sistema.

    - Registro de ventas y actualización automática del inventario.
    - Consulta de productos en el catálogo.
    - Comunicación entre el backend y la base de datos administrada mediante Supabase.
- **Framework:** Supertest / Postman
- **Alcance:** APIs, servicios y base de datos
- **Frecuencia:** En cada Pull Request antes de realizar el merge al repositorio principal.

Ejemplo de prueba de integración:

```
// api.integration.test.js
describe('User API Integration', () => {
 test('POST /api/users should create user', async () => {
   const response = await request(app)
     .post('/api/users')
     .send({
       name: 'John Doe',
       email: 'john@example.com',
       password: 'securePass123'
     });
   expect(response.status).toBe(201);
   expect(response.body).toHaveProperty('id');
   expect(response.body.email).toBe('john@example.com');
 });
 test('GET /api/users/:id should return user', async () => {
```

```
  const userId = 1;
  const response = await request(app)
    .get(`/api/users/${userId}`);
  expect(response.status).toBe(200);
  expect(response.body).toHaveProperty('name');
 });
});
```

3.3.3. Pruebas End-To-End (E2E)

Las pruebas End-to-End (E2E) permiten validar el funcionamiento completo del sistema desde la perspectiva del usuario final. Estas pruebas simulan la interacción real de los usuarios con la plataforma para verificar que los flujos principales del sistema funcionen correctamente.

En el sistema CobriXS se evaluarán flujos como:

- Inicio de sesión de usuarios.
- Registro completo de una venta en el panel de cobro.
- Consulta de productos en el catálogo.
- Actualización automática del inventario después de una venta.
- Registro de pagos a proveedores.

- Framework: Cypress y Selenium
- Alcance: Flujos completos de usuario
- Frecuencia: Antes de cada versión o release del sistema.

Ejemplo de prueba E2E (Cypress):

```
// login.e2e.spec.js
describe('User Login Flow', () => {
 it('should login successfully with valid credentials', () => {
   cy.visit('/login');
   cy.get('#email')
     .type('user@example.com');
   cy.get('#password')
     .type('password123');
   cy.get('button[type="submit"]')
     .click();
   cy.url().should('include', '/dashboard');
   cy.contains('Welcome back').should('be.visible');
 });
 it('should show error with invalid credentials', () => {
   cy.visit('/login');
   cy.get('#email')
     .type('wrong@example.com');
   cy.get('#password')
     .type('wrongpass');
   cy.get('button[type="submit"]')
     .click();
   cy.contains('Invalid credentials')
     .should('be.visible');
 });
});
```

3.4. Criterios De Aceptación

Los criterios de aceptación definen las condiciones que deben cumplirse para considerar que el sistema CobriXS ha superado correctamente el proceso de pruebas y que sus funcionalidades operan de forma estable.

Para que las pruebas del sistema se consideren exitosas, deberán cumplirse los siguientes criterios:

- Todas las pruebas unitarias deben pasar (100%), garantizando que las funciones individuales del sistema, como validaciones de formularios, cálculos de ventas, gestión de usuarios y actualización del inventario, funcionen correctamente de manera aislada.
- La cobertura de código deberá ser igual o superior al 70%, asegurando que una parte significativa de la lógica del sistema ha sido evaluada mediante pruebas automatizadas.
- Todas las pruebas de integración críticas deberán aprobarse, verificando que la comunicación entre el frontend desarrollado en React, el backend en Node.js y la base de datos administrada mediante Supabase funcione correctamente.
- Al menos cinco flujos principales del sistema deberán funcionar correctamente en las pruebas End-to-End (E2E). Estos flujos incluyen procesos clave como el inicio de sesión, el registro de una venta, la actualización automática del inventario, la consulta de productos y el registro de pagos a proveedores.

- No deberán existir errores críticos o bloqueantes, es decir, fallos que impidan el uso normal de las funcionalidades principales del sistema o que afecten la integridad de los datos almacenados.

3.5. Ambiente De Pruebas

El ambiente de pruebas define los entornos donde se ejecutarán las diferentes pruebas del sistema CobriXS, permitiendo validar su funcionamiento antes de su implementación final.

- **Ambiente local:** Cada desarrollador podrá ejecutar el sistema en su entorno local utilizando contenedores Docker para simular el backend del sistema. En este entorno se realizarán pruebas unitarias y algunas pruebas de integración iniciales.
- **Ambiente de staging:** Se dispondrá de un entorno de pruebas previo a producción donde el sistema será desplegado para realizar pruebas más completas. Este entorno permitirá verificar la interacción entre el frontend, el backend y los servicios en la nube proporcionados por Supabase antes de su liberación final.
- **Datos de prueba:** Para la ejecución de pruebas se utilizará un conjunto de datos sintéticos o anonimizados, incluyendo productos, usuarios, proveedores y registros de ventas simulados. Esto permite validar el comportamiento del sistema sin utilizar información real de negocios.

3.6. Aprobaciones Y Responsables

La organización de las pruebas establece los responsables de cada tipo de prueba y la frecuencia con la que se ejecutarán durante el desarrollo del sistema.

Tipo de Prueba	Responsable	Frecuencia
Pruebas Unitarias	Desarrolladores	En cada commit
Pruebas de Integración	Desarrolladores / QA	En cada Pull Request
Pruebas End-to-End (E2E)	QA Lead / Equipo de pruebas	Antes de cada release

4. Casos De Pruebas

4.1. Formato De Caso De Pruebas

CP-001: Registro de Usuario Exitoso

- **ID:** CP-001
- **Nombre:** Registro de Usuario Exitoso
- **Módulo / Feature:** Gestión de Usuarios
- **Prioridad:** Alta
- **Precondiciones:**
    - La aplicación CobriXS se encuentra en ejecución.
    - El administrador tiene acceso al módulo de gestión de usuarios.
    - La base de datos Supabase está disponible y conectada al sistema.
- **Pasos:**
    - Ingresar al sistema con una cuenta de administrador.
    - Acceder al módulo de gestión de usuarios.

- Seleccionar la opción "Crear nuevo usuario".
- Ingresar los datos solicitados.
- Seleccionar el rol del usuario.
- Guardar el registro.

- **Datos de entrada:**
  - Nombre: Maria Pérez
  - Correo:maria@cobrixs.com
  - Contraseña: Pass123
  - Rol: Cajero
- **Resultado esperado:**

  El sistema registra correctamente el nuevo usuario en la base de datos y muestra un mensaje de confirmación indicando que el usuario fue creado exitosamente.
- **Resultado obtenido:**

  Pendiente
- **Estado:**

  Pendiente

CP-002: Venta de Producto Exitosa

- **ID:** CP-002
- **Nombre:** Venta de Producto Exitosa
- **Módulo / Feature:** Ventas
- **Prioridad:** Alta

- **Precondiciones:**
    - El cajero está autenticado en el sistema.
    - El producto existe en el inventario.
    - El sistema tiene conexión con la base de datos.
- **Pasos:**
    - El usuario accede al módulo de ventas.
    - Escanea o selecciona el producto desde el catálogo.
    - El sistema muestra el producto en la lista de compra.
    - El usuario confirma la venta.
- **Datos de entrada:**
    - Producto: Refresco 600 ml
    - Cantidad: 1
    - Monto recibido: $50.00
- **Resultado esperado:**

  El sistema registra la venta, genera el ticket correspondiente y guarda la transacción en la base de datos.
- **Resultado obtenido:**

  Pendiente
- **Estado:**

  Pendiente

CP-003: Actualización de Inventario Automática

- **ID:** CP-003

- **Nombre:** Actualización de Inventario Automática
- **Módulo / Feature:** Inventario
- **Prioridad:** Alta
- **Precondiciones:**
    - Existe un producto registrado en el inventario.
    - El producto tiene stock disponible.
    - Se realiza una venta del producto.
- **Pasos:**
    - Realizar una venta de un producto desde el módulo de ventas.
    - Finalizar la venta.
    - Acceder al módulo de inventario.
    - Consultar el stock del producto vendido.
- **Datos de entrada:**
    - Producto: Galletas
    - Cantidad vendida: 1
    - Monto recibido:$40.00
- **Resultado esperado:**

  El sistema disminuye automáticamente la cantidad del producto en el inventario después de realizar la venta.
- **Resultado obtenido:**

  Pendiente
- **Estado:**

  Pendiente

CP-004: Registro de Pago a Proveedor

- **ID:** CP-004
- **Nombre:** Registro de Pago a Proveedor
- **Módulo / Feature:** Pagos a Proveedores
- **Prioridad:** Media
- **Precondiciones:**
  - El usuario está autenticado en el sistema.
  - El proveedor está registrado previamente en el sistema.
- **Pasos:**
  - Acceder al módulo de pagos a proveedores.
  - Seleccionar el proveedor correspondiente.
  - Ingresar el monto del pago.
  - Confirmar el registro del pago.
- **Datos de entrada:**

  Proveedor: Distribuidora ABC

  Monto: $2000.00
- **Resultado esperado:**

  El sistema registra el pago correctamente y actualiza el historial de pagos del proveedor.
- **Resultado obtenido:**

  Pendiente

- **Estado:**

  Pendiente

CP-005: Registro con Correo Existente

- **ID:** CP-005
- **Nombre:** Registro con Correo Existente
- **Módulo / Feature:** Gestión de Usuarios
- **Prioridad:** Alta
- **Precondiciones:**
  - Existe un usuario previamente registrado con el mismo correo electrónico.
  - El administrador intenta registrar un nuevo usuario con ese correo.
- **Pasos:**
  - Acceder al módulo de gestión de usuarios.
  - Intentar registrar un usuario con un correo ya existente.
  - Confirmar el registro.
- **Datos de entrada:**

  Correo: maria@cobrixs.com
- **Resultado esperado:**

  El sistema muestra un mensaje de error indicando que el correo ya está registrado y no permite completar el registro.
- **Resultado obtenido:**

  Pendiente

- **Estado:**

  Pendiente

CP-006: Venta sin Productos

- **ID:** CP-006
- **Nombre:** Venta sin Productos
- **Módulo / Feature:** Ventas
- **Prioridad:** Media
- **Precondiciones:**
  - El usuario está autenticado en el sistema.
  - El módulo de ventas está abierto.
- **Pasos:**
  - Acceder al módulo de ventas.
  - Intentar finalizar la venta sin agregar productos.
- **Datos de entrada:**

  Ninguno
- **Resultado esperado:**

  El sistema muestra un mensaje de error indicando que no se puede completar la venta sin productos.
- **Resultado obtenido:**

  Pendiente

- **Estado:**

  Pendiente

CP-007: Pago a Proveedor con Monto Negativo

- **ID:** CP-007
- **Nombre:** Pago a Proveedor con Monto Negativo
- **Módulo / Feature:** Pagos a Proveedores
- **Prioridad:** Media
- **Precondiciones:**
  - El usuario está autenticado en el sistema.
  - El proveedor existe en el sistema.
- **Pasos:**
  - Acceder al módulo de pagos a proveedores.
  - Seleccionar un proveedor.
  - Ingresar un monto negativo.
  - Intentar guardar el registro.
- **Datos de entrada:**

  Monto: -500
- **Resultado esperado:**

  El sistema muestra un mensaje de error indicando que el monto no puede ser negativo y no registra el pago.

- **Resultado obtenido:**

  Pendiente

- **Estado:**

  Pendiente

CP-008: Venta con Último Producto en Inventario

- **ID:** CP-008
- **Nombre:** Venta con Último Producto en Inventario
- **Módulo / Feature:** Ventas / Inventario
- **Prioridad:** Alta
- **Precondiciones:**
  - Existe un producto con stock igual a 1.
  - El usuario está autenticado en el sistema.
- **Pasos:**
  - Acceder al módulo de ventas.
  - Seleccionar el producto con stock mínimo.
  - Confirmar la venta.
  - Revisar el inventario del producto.
- **Datos de entrada:**
  - Producto: Leche
  - Cantidad: 1
  - Monto recibido: $30.00

- **Resultado esperado:**

  El sistema completa la venta, reduce el stock a 0 y muestra una alerta indicando que el producto está agotado.

- **Resultado obtenido:**

  Pendiente

- **Estado:**

  Pendiente

CP-009: Registro de Producto con Nombre Muy Largo

- **ID:** CP-009
- **Nombre:** Registro de Producto con Nombre Muy Largo
- **Módulo / Feature:** Inventario
- **Prioridad:** Baja
- **Precondiciones:**
  - El administrador está autenticado en el sistema.
  - El módulo de inventario está disponible.
- **Pasos:**
  - Acceder al módulo de inventario.
  - Seleccionar la opción "Agregar producto".
  - Ingresar un nombre de producto extremadamente largo (más 50 y menos de 100 caracteres).
  - Intentar guardar el producto.

- **Datos de entrada:**

  Nombre del producto: Texto mayor a la longitud permitida (más 50 y menos de 100 caracteres).

- **Resultado esperado:**

  El sistema muestra una notificación indicando que el nombre del producto es más largo de lo común.

- **Resultado obtenido:**

  Pendiente

- **Estado:**

  Pendiente

CP-010: Venta Actualiza Inventario y Caja

- **ID:** CP-010
- **Nombre:** Venta Actualiza Inventario y Caja
- **Módulo / Feature:** Integración Ventas – Inventario – Caja
- **Prioridad:** Alta
- **Precondiciones:**
  - El producto existe en el inventario.
  - El usuario está autenticado en el sistema.
  - El sistema tiene conexión con la base de datos.
- **Pasos:**
  - Acceder al módulo de ventas.

- Seleccionar un producto.
- Confirmar la venta.
- Revisar el inventario y el registro de caja.

- **Datos de entrada:**
  - Producto: Pan
  - Cantidad: 2
  - Monto recibido: $60.00

- **Resultado esperado:**

  El sistema registra la venta, actualiza el inventario y aumenta el fondo de caja con el monto correspondiente.

- **Resultado obtenido:**

  Pendiente

- **Estado:**

  Pendiente

4.2. Ejemplo de Caso de Pruebas

CP-001: Registro de Usuario Exitoso

CP-002: Venta de Producto Exitosa

CP-003: Actualización de Inventario Automática

CP-004: Registro de Pago a Proveedor

CP-005: Registro con Correo Existente CP-006: Venta sin Productos

CP-007: Pago a Proveedor con Monto Negativo

CP-008: Venta con Último Producto en Inventario

CP-009: Registro de Producto con Nombre Muy Largo

CP-010: Venta Actualiza Inventario y Caja

4.3. Plantilla de Casos de Prueba

Casos De Éxito (Happy Path)

CP-001: Registro De Usuario Exitoso

Campo	Descripción
ID	CP-001
Módulo	Gestión de Usuarios
Prioridad	Alta
Precondiciones	1. La aplicación CobriXS se encuentra en ejecución. 2. El administrador tiene acceso al módulo de gestión de usuarios. 3. La base de datos Supabase está disponible y conectada al sistema.
Pasos	1. Ingresar al sistema con una cuenta de administrador. 2. Acceder al módulo de gestión de usuarios. 3. Seleccionar la opción "Crear nuevo usuario". 4. Ingresar los datos solicitados. 5. Seleccionar el rol del usuario. 6. Guardar el registro.
Resultado esperado	El sistema registra correctamente el nuevo usuario en la base de datos y muestra un mensaje de confirmación indicando que el usuario fue creado exitosamente.
Estado	Pendiente

CP-002: Venta De Producto Exitosa

Campo	Descripción
ID	CP-002
Módulo	Ventas

Prioridad	Alta
Precondiciones	1. El cajero está autenticado en el sistema. 2. El producto existe en el inventario. 3. El sistema tiene conexión con la base de datos.
Pasos	1. El usuario accede al módulo de ventas. 2. Escanea o selecciona el producto desde el catálogo. 3. El sistema muestra el producto en la lista de compra. 4. El usuario confirma la venta.
Resultado esperado	El sistema registra la venta, genera el ticket correspondiente y guarda la transacción en la base de datos.
Estado	Pendiente

CP-003: Actualización De Inventario Automática

Campo	**Descripción**
ID	CP-003
Módulo	Inventario
Prioridad	Alta
Precondiciones	1. Existe un producto registrado en el inventario. 2. El producto tiene stock disponible. 3. Se realiza una venta del producto.
Pasos	1. Realizar una venta de un producto desde el módulo de ventas. 2. Finalizar la venta. 3. Acceder al módulo de inventario. 4. Consultar el stock del producto vendido.
Resultado esperado	El sistema disminuye automáticamente la cantidad del producto en el inventario después de realizar la venta.
Estado	Pendiente

CP-004: Registro De Pago A Proveedor

Campo	**Descripción**
ID	CP-004

Módulo	Pago a Proveedores
Prioridad	Media
Precondiciones	1. El usuario está autenticado en el sistema. 2. El proveedor está registrado previamente en el sistema.
Pasos	1. Acceder al módulo de pagos a proveedores. 2. Seleccionar el proveedor correspondiente. 3. Ingresar el monto del pago. 4. Confirmar el registro del pago.
Resultados Esperados	El sistema registra el pago correctamente y actualiza el historial de pagos del proveedor.
Estado	Pendiente

Casos De Error (Validaciones)

CP-005: Registro Con Correo Existente

Campo	**Descripción**
ID	CP-005
Módulo	Gestión de Usuarios
Prioridad	Alta
Precondiciones	1. Existe un usuario previamente registrado con el mismo correo electrónico. 2. El administrador intenta registrar un nuevo usuario con ese correo.
Pasos	1. Acceder al módulo de gestión de usuarios. 2. Intentar registrar un usuario con un correo ya existente. 3. Confirmar el registro.
Resultado esperado	El sistema muestra un mensaje de error indicando que el correo ya está registrado y no permite completar el registro.
Estado	Pendiente

CP-006: Venta Sin Productos

Campo	Descripción
ID	CP-006
Módulo	Ventas
Prioridad	Media
Precondiciones	1. El usuario está autenticado en el sistema. 2. El módulo de ventas está abierto.
Pasos	1. Acceder al módulo de ventas. 2. Intentar finalizar la venta sin agregar productos.
Resultado esperado	El sistema muestra un mensaje de error indicando que no se puede completar la venta sin productos.
Estado	Pendiente

CP-007: Pago A Proveedor Con Monto Negativo

Campo	Descripción
ID	CP-007
Módulo	Pagos a proveedores
Prioridad	Media
Precondiciones	1. El usuario está autenticado en el sistema. 2. El proveedor existe en el sistema.
Pasos	1. Acceder al módulo de pagos a proveedores. 2. Seleccionar un proveedor. 3. Ingresar un monto negativo. 4. Intentar guardar el registro.
Resultado esperado	El sistema muestra un mensaje de error indicando que el monto no puede ser negativo y no registra el pago.
Estado	Pendiente

Casos Límite (Edge Cases)

CP-008: Venta Con Último Producto En Inventario

Campo	Descripción
ID	CP-008
Módulo	Ventas / Inventario
Prioridad	Alta
Precondiciones	1. Existe un producto con stock igual a uno. 2. El usuario está autenticado en el sistema.
Pasos	1. Acceder al módulo de ventas. 2. Seleccionar el producto con stock mínimo. 3. Confirmar la venta. 4. Revisar el inventario del producto.
Resultado esperado	El sistema completa la venta, reduce el stock a 0 y muestra una alerta indicando que el producto está agotado.
Estado	Pendiente

CP-009: Registro De Producto Con Nombre Muy Largo

Campo	Descripción
ID	CP-009
Módulo	Inventario
Prioridad	Baja
Precondiciones	1. El administrador está autenticado en el sistema. 2. El módulo de inventario está disponible.
Pasos	1. Acceder al módulo de inventario. 2. Seleccionar la opción "Agregar producto". 3. Ingresar un nombre de producto extremadamente largo (más 50 y menos de 100 caracteres). 4. Intentar guardar el producto.
Resultado esperado	El sistema muestra una notificación indicando que el nombre del producto es más largo de lo común.
Estado	Pendiente

Casos De Integración

CP-010: Venta Actualiza Inventario Y Caja

Campo	Descripción
ID	CP-010
Módulo	Integración Ventas – Inventario – Caja
Prioridad	Alta
Precondiciones	1. El producto existe en el inventario. 2. El usuario está autenticado en el sistema. 3. El sistema tiene conexión con la base de datos.
Pasos	1. Acceder al módulo de ventas. 2. Seleccionar un producto. 3. Confirmar la venta. 4. Revisar el inventario y el registro de caja.
Resultado esperado	El sistema registra la venta, actualiza el inventario y aumenta el fondo de caja con el monto correspondiente.
Estado	Pendiente

5. Flujo De Trabajo Para Control De Versiones

5.1. Estrategia de Branching Seleccionada

Estrategia elegida: GitHub Flow

Justificación

Se eligió GitHub Flow porque el proyecto CobriXS es una aplicación web moderna que se desarrolla de manera incremental usando metodología Scrum. Esta estrategia permite trabajar con ramas pequeñas para cada funcionalidad y facilita la integración continua mediante Pull Requests y revisiones de código.

Además, GitHub Flow se adapta bien al flujo de trabajo del equipo porque:

- Permite desarrollar features independientes sin afectar la versión estable.
- Facilita la colaboración entre desarrolladores mediante revisiones.
- Funciona bien con CI/CD usando GitHub Actions.
- Reduce la complejidad comparado con Git Flow.

Esta estrategia es adecuada para CobriXS, ya que el sistema está organizado en módulos como:

- Inicio (Dashboard)
- Usuarios
- Productos
- Inventari
- Panel de Cobro
- Pagos a Proveedores
- Reportes
- Configuración

Cada módulo puede desarrollarse en ramas independientes y luego integrarse a la rama principal mediante Pull Requests.

5.2. Estructura De Ramas

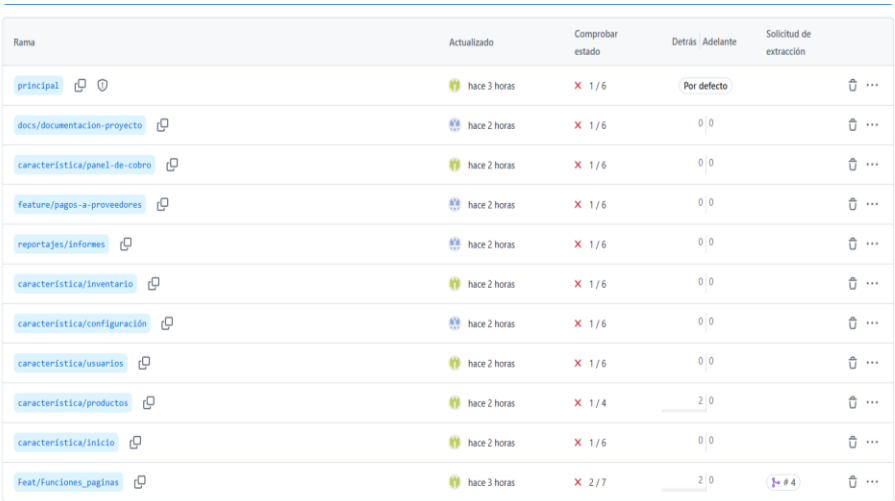
Documenta la estructura completa de ramas que usarás:

Ejemplo para Git Flow:

main (producción)

│

```
├── feature/inicio
├── feature/usuarios
├── feature/productos
├── feature/inventario
├── feature/panel-de-cobro
├── feature/pagos-a-proveedores
├── feature/reportes
├── feature/configuracion
│
├── bugfix/error-calculo-en-panel-de-cobro
├── bugfix/error-alertas-de-stock-en-inventario
├── bugfix/error-accion-editar-en-usuarios
│
├── hotfix/error-nuevo-usuario-en-usuarios
├── hotfix/error-procesar-pago-en-panel-de-cobro
│
└── docs/documentacion-proyecto
```



Rama	Actualizado	Comprobar estado	Detrás	Adelante	Solicitud de extracción
principal	hace 3 horas	1 / 6	Por defecto		
docs/documentacion-proyecto	hace 2 horas	1 / 6	0	0	
característica/panel-de-cobro	hace 2 horas	1 / 6	0	0	
feature/pagos-a-proveedores	hace 2 horas	1 / 6	0	0	
reportajes/informes	hace 2 horas	1 / 6	0	0	
característica/inventario	hace 2 horas	1 / 6	0	0	
característica/configuración	hace 2 horas	1 / 6	0	0	
característica/usuarios	hace 2 horas	1 / 6	0	0	
característica/productos	hace 2 horas	1 / 4	2	0	
característica/inicio	hace 2 horas	1 / 6	0	0	
Feat/Funciones_paginas	hace 3 horas	2 / 7	2	0	# 4

Ilustración 1. Ramas

Rama	Propósito
main	Contiene la versión estable del sistema en producción
feature/	Desarrollo de nuevas funcionalidades o módulos
bugfix/	Corrección de errores detectados durante pruebas
hotfix/	Correcciones urgentes directamente en producción
docs/	Cambios relacionados únicamente con la documentación

5.3 Nomenclatura De Ramas

Tipo de rama	Ejemplo	Uso
main	main	Rama principal que contiene la versión estable del sistema
feature	feature/productos	Desarrollo de nuevas funcionalidades o módulos
bugfix	bugfix/error-alertas-de-stock-en-inventario	Corrección de errores detectados durante pruebas
hotfix	hotfix/error-procesar-pago-en-panel-de-cobro	Correcciones urgentes en producción
docs	docs/documentacion-proyecto	Cambios o mejoras en la documentación

Reglas Para Nombrar Ramas

Para mantener consistencia en el repositorio se aplicarán las siguientes reglas:

- Utilizar minúsculas en todos los nombres de ramas.
- Separar palabras mediante guiones (-).
- Incluir el módulo o funcionalidad afectada en el nombre de la rama.
- Usar prefijos que indiquen el tipo de cambio (feature, bugfix, hotfix, docs).

5.4. Convención De Commits

Para mantener un historial de cambios claro se utilizará la convención **Conventional Commits**.

Formato: tipo(modulo): descripcion del cambio

Tipo	Uso
feat	Nueva funcionalidad
fix	Corrección de errores
docs	Cambios en documentación
style	Cambios de formato
refactor	Mejora interna del código
test	Agregar o modificar pruebas
chore	Cambios en configuración o dependencias

Ejemplos Aplicados A Cobrixs

- **feat(productos):** agregar registro de nuevos productos
- **feat(inventario):** implementar alertas de stock bajo
- **fix(panel-de-cobro):** corregir cálculo total de venta
- **docs(readme):** actualizar guía de instalación
- **test(ventas):** agregar pruebas para proceso de venta

5.5. Proceso De Pull Request

El proceso de integración de cambios se realizará mediante Pull Requests en GitHub.

Flujo de trabajo:

1. Crear una nueva rama desde main

git checkout main

git pull origin main

git checkout -b feature/productos

2. Realizar cambios en el código.
3. Guardar los cambios mediante commits.
4. Subir la rama al repositorio remoto.

git push origin feature/productos

5. Crear un Pull Request en GitHub.
6. El Pull Request será revisado por otro miembro del equipo.
7. Si las pruebas y revisiones son correctas, se realiza el merge a la rama main.

5.6. Proceso De Pull Request

Configura esta plantilla en .github/PULL_REQUEST_TEMPLATE.md:

Pull Request - CobriXS

Descripción

Describe brevemente los cambios realizados en este Pull Request.

Ejemplo:

Se agregó el módulo de gestión de usuarios permitiendo crear, editar y eliminar usuarios.

Tipo de cambio

Marca con una X el tipo de cambio:

- [ ] Bug fix (corrección de error)

- [ ] Nueva feature (nueva funcionalidad)

- [ ] Breaking change (cambio que afecta funcionalidades existentes)

- [ ] Documentación

¿Cómo se ha probado?

Describe las pruebas realizadas para validar los cambios.

- [ ] Tests unitarios

- [ ] Tests de integración

- [ ] Pruebas manuales

Checklist

Antes de enviar el Pull Request verifica lo siguiente:

- [ ] Mi código sigue las guías de estilo (ESLint / Prettier)

- [ ] He revisado mi propio código

- [ ] He comentado partes complejas del código

- [ ] He actualizado la documentación si fue necesario

- [ ] Mis cambios no generan warnings

- [ ] He agregado tests para las nuevas funcionalidades

- [ ] Todos los tests pasan correctamente en local

Screenshots (si aplica)

Agrega capturas si el cambio afecta la interfaz del sistema CobriXS.

Issues relacionados

Closes #123

5.7 Reglas De Protección De Ramas (Branch Protection Rules)

Para garantizar la estabilidad del sistema se aplicarán reglas de protección en la rama main dentro del repositorio de GitHub.

Configuración aplicada:

- No se permiten push directos a la rama main.
- Todos los cambios deben realizarse mediante Pull Request.
- Se requiere al menos una revisión de código antes de aprobar el cambio.
- Deben ejecutarse correctamente los status checks del pipeline CI/CD antes de realizar el merge.
- Todas las conversaciones del Pull Request deben estar resueltas antes de integrarse.
- No se permite realizar force push sobre la rama main.

Estas reglas permiten mantener la calidad del código, mejorar la colaboración entre los miembros del equipo y reducir el riesgo de errores en producción.

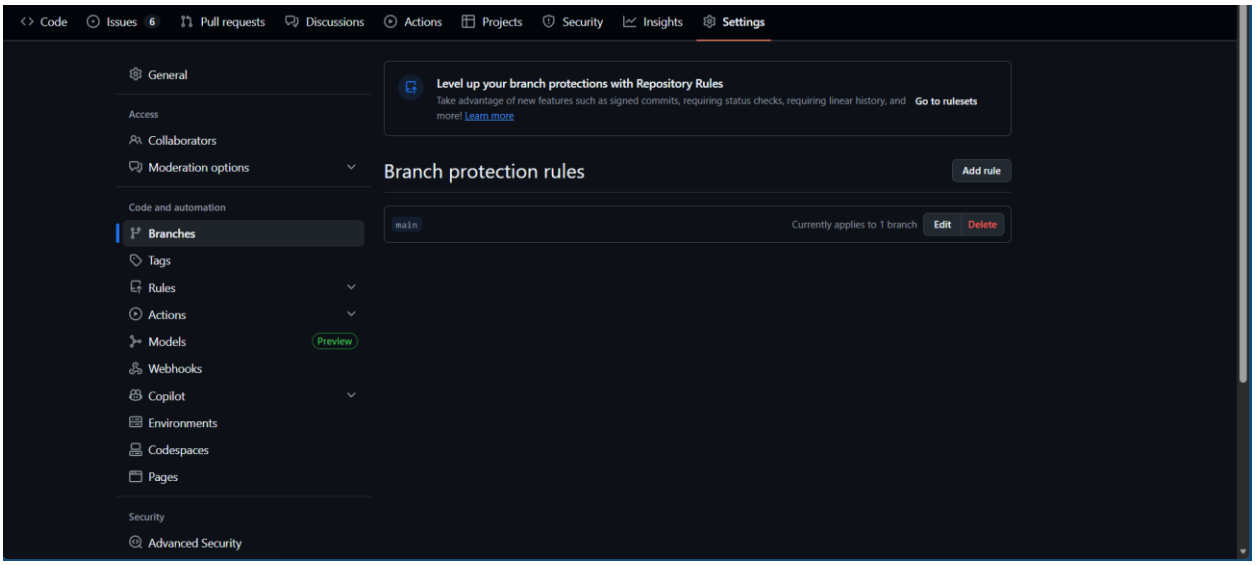


Ilustración 2. Branch protection rules

5.8 Flujo De Trabajo Diario

El equipo de desarrollo del proyecto CobriXS seguirá un flujo de trabajo basado en GitHub Flow, que permite desarrollar nuevas funcionalidades de forma organizada y segura sin afectar la versión estable del sistema.

El proceso de trabajo diario se realiza mediante la creación de ramas independientes para cada funcionalidad, corrección o mejora, las cuales posteriormente se integran a la rama principal mediante Pull Requests.

Inicio Del Desarrollo

Al iniciar una nueva funcionalidad, el desarrollador debe actualizar la rama principal y crear una nueva rama de trabajo.

```
git checkout main

git pull origin main
```

```
git checkout -b feature/mi-nueva-funcionalidad
```

Esta nueva rama se utilizará para desarrollar la funcionalidad correspondiente a uno de los módulos del sistema, por ejemplo:

feature/productos

feature/inventario

feature/panel-de-cobro

Durante El Desarrollo

Mientras se desarrolla la funcionalidad, los cambios se guardan mediante commits siguiendo la convención establecida para el proyecto.

```
git add .
git commit -m "feat(productos): agregar registro de productos"
```

Posteriormente se sube la rama al repositorio remoto:

```
git push origin feature/mi-nueva-funcionalidad
```

Finalización De La Funcionalidad

Una vez que el desarrollo ha terminado, el programador debe realizar los siguientes pasos:

1. Crear un Pull Request en GitHub desde la rama de trabajo hacia la rama main.
2. Un miembro del equipo realiza la revisión de código.
3. Se ejecutan las pruebas automáticas del sistema.

4. Si todo es correcto, se realiza el merge del Pull Request.
5. Finalmente se elimina la rama utilizada.

Corrección De Errores

Cuando se detecta un error durante el desarrollo o pruebas, se crea una rama **bugfix**.

Ejemplo:

git checkout -b bugfix/error-alertas-de-stock-en-inventario

Una vez solucionado el error, se sigue el mismo proceso de Pull Request y revisión de código.

Correcciones Urgentes

Si se detecta un error crítico en producción, se crea una rama hotfix.

Ejemplo:

git checkout -b hotfix/error-procesar-pago-en-panel-de-cobro

Después de corregir el problema, el cambio se integra a la rama main mediante un Pull Request.

6. Estrategia De Despliegue

6.1 Estrategia Seleccionada

Estrategia elegida: Canary Deployment

Justificación

La estrategia Canary Deployment permite desplegar nuevas versiones del sistema de manera gradual a un pequeño grupo de usuarios antes de liberarlas completamente.

Esto reduce riesgos en caso de errores y permite detectar problemas antes de afectar a todos los usuarios del sistema.

Para CobriXS, esta estrategia es importante porque el sistema gestiona:

- ventas
- inventario
- pagos a proveedores

Por lo tanto, cualquier error podría afectar directamente las operaciones del negocio.

6.2. Ambientes De Despliegue

Ambiente	Propósito	URL	Deploy
Development	Desarrollo local	localhost:3000	Manual
Staging	Pruebas del sistema	staging.cobrixs.com	Automático
Production	Sistema en uso	cobrixs.com	Automático

- **Development:** Es el entorno utilizado por los desarrolladores para implementar y probar nuevas funcionalidades del sistema CobriXS de manera local antes de integrarlas al repositorio principal.
- **Staging:** Es un entorno de pruebas que replica el comportamiento del sistema en producción. Aquí se realizan pruebas funcionales y de integración antes de liberar una nueva versión del sistema.

- **Production:** Es el entorno donde el sistema CobriXS se encuentra disponible para los usuarios finales y donde se ejecuta la versión estable del sistema.

6.3. Proceso De Despliegue Detallado

Staging

1. Merge de feature a main
2. GitHub Actions ejecuta pipeline
3. Ejecuta tests unitarios
4. Ejecuta pruebas de integración
5. Build del proyecto
6. Deploy automático en Vercel
7. QA prueba funcionalidades

Producción

1. Pull Request aprobado
2. Pipeline CI/CD ejecutad
3. Tests completos
4. CodeClimate analiza calidad de código
5. Build de aplicación
6. Deploy Canary
7. Monitoreo inicial

6.4. Configuración Del Pipeline De Despliegue

Ejemplo de workflow de despliegue (.github/workflows/deploy.yml):

Archivo:

.github/workflows/deploy.yml

Ejemplo adaptado a CobriXS:

```
name: Deploy CobriXS
on:
  push:
    branches: [ main ]
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Descargar repositorio
        uses: actions/checkout@v3
      - name: Instalar dependencias
        run: npm install
      - name: Ejecutar tests
        run: npm test
      - name: Construir proyecto
        run: npm run build
      - name: Deploy CobriXS
        run: echo "Deploy CobriXS to production"
```

6.5. Plan De Rollback

En caso de que un despliegue en producción genere fallas o afecte el funcionamiento del sistema **CobriXS**, se aplicará un proceso de rollback para restaurar la última versión estable del sistema.

Política De Rollback

- **Tiempo máximo para ejecutar rollback:** 5 minutos.
- **Responsable:** Tech Lead o responsable del despliegue del proyecto CobriXS.
- **Condiciones que activan rollback:**
  - Tasa de errores (**Error Rate**) mayor al **1%**.
  - Latencia del sistema mayor al **doble del valor normal esperado**.
  - Fallas críticas en funcionalidades principales del sistema, como gestión de usuarios, acceso al sistema o procesamiento de datos.

Procedimiento De Rollback

1. Detectar el problema mediante las herramientas de monitoreo.
2. Notificar al responsable del despliegue.
3. Revertir el sistema a la última versión estable disponible en el repositorio.
4. Verificar que el sistema vuelva a operar correctamente.
5. Registrar el incidente para su posterior análisis y corrección.

6.6. Métricas de Monitoreo

Después de cada despliegue del sistema CobriXS, se monitorearán diferentes métricas para garantizar la estabilidad, disponibilidad y rendimiento del sistema.

Métrica	Umbral Normal	Acción si Excede
Error Rate	< 0.1%	Ejecutar rollback inmediato
Latencia P95	< 500 ms	Investigar causa y generar alerta
CPU Usage	< 70%	Escalar recursos del servidor

Memory Usage	< 80%	Escalar recursos o reiniciar servicio

7. Repositorio Configurado

7.1 Estructura Del Repositorio

El repositorio del sistema CobriXS debe mantener una estructura organizada que facilite el desarrollo, mantenimiento y escalabilidad del sistema.

```
cobrixs/
COBRIX-V_2/
├── .github/
│   └── workflows/
│       ├── ci-cd.yml
│       ├── deploy.yml
│       └── PULL_REQUEST_TEMPLATE.md
├── public/
│   └── vite.svg
├── src/
│   ├── components/
│   │   └── ProtectedRoute.tsx
│   ├── context/
│   │   └── SettingsContext.tsx
│   ├── hooks/
│   │   ├── useAuth.ts
│   │   ├── useCashier.ts
│   │   ├── useInventory.ts
│   │   ├── useProducts.ts
│   │   └── useUsers.ts
```

```
│   ├── i18n/
│   │   ├── en.ts
│   │   └── es.ts
│   ├── layout/
│   │   ├── AppLayout.jsx
│   │   ├── DashboardLayout.tsx
│   │   ├── Sidebar.tsx
│   │   └── Topbar.tsx
│   ├── pages/
│   │   ├── Cashier.tsx
│   │   ├── Dashboard.tsx
│   │   ├── Inventory.tsx
│   │   ├── Login.tsx
│   │   ├── Products.tsx
│   │   ├── Register.tsx
│   │   ├── Reports.tsx
│   │   ├── Settings.tsx
│   │   ├── SupplierPayments.tsx
│   │   ├── TestSupabase.tsx
│   │   └── Users.tsx
│   ├── services/
│   │   ├── authService.ts
│   │   ├── cashSession.service.ts
│   │   ├── inventory.service.ts
│   │   ├── products.service.ts
│   │   ├── providerPayments.service.ts
│   │   ├── providers.service.ts
```

```
│   │   ├── reports.service.ts
│   │   ├── sales.service.ts
│   │   └── users.service.ts
│   ├── Styles/
│   │   └── layout.css
│   ├── types/
│   │   └── database.ts
│   ├── utils/
│   │   ├── App.tsx
│   │   ├── index.css
│   │   ├── main.tsx
│   │   └── router.tsx
│   ├── lib/
│   │   └── supabase.ts
│   └── index.tsx
├── supabase/
│   ├── fix_register.sql
│   └── schema.sql
├── tests/
│   └── (unit y e2e tests)
├── .env
├── .env.example
├── .gitignore
├── docker-compose.yml
├── Dockerfile
├── eslint.config.js
├── tsconfig.app.json
```

```
├── tsconfig.json
├── tsconfig.node.json
├── vite.config.ts
├── package-lock.json
├── package.json
├── index.html
└── README.md
```

7.2 README.Md Completo

El archivo README.md del repositorio de CobriXS debe contener la documentación básica necesaria para comprender, instalar y ejecutar el proyecto.

El README deberá incluir las siguientes secciones:

- Título y descripción del proyecto
- Badges de CI/CD, cobertura y licencia
- Tabla de contenidos
- Requisitos previos
- Instrucciones de instalación
- Instrucciones para ejecutar el sistema localmente
- Instrucciones para ejecutar pruebas
- Estructura del proyecto
- Stack tecnológico
- Variables de entorno
- Guía de contribución (Pull Requests)
- Licencia

- Autores

README.md

React + TypeScript + Vite

This template provides a minimal setup to get React working in Vite with HMR and some ESLint rules.

Currently, two official plugins are available:

- [@vitejs/plugin-react](https://github.com/vitejs/vite-plugin-react/blob/main/packages/plugin-react) uses [Babel](https://babeljs.io/) (or [oxc](https://oxc.rs) when used in [rolldown-vite](https://vite.dev/guide/rolldown)) for Fast Refresh

- [@vitejs/plugin-react-swc](https://github.com/vitejs/vite-plugin-react/blob/main/packages/plugin-react-swc) uses [SWC](https://swc.rs/) for Fast Refres

React Compiler

The React Compiler is not enabled on this template because of its impact on dev & build performances. To add it, see [this documentation](https://react.dev/learn/react-compiler/installation).

Expanding the ESLint configuration

If you are developing a production application, we recommend updating the configuration to enable type-aware lint rules:

```js

export default defineConfig([

  globalIgnores(['dist']),

  {

    files: ['**/*.{ts,tsx}'],

    extends: [
```

```
    // Other configs...
    // Remove tseslint.configs.recommended and replace with this
    tseslint.configs.recommendedTypeChecked,
    // Alternatively, use this for stricter rules
    tseslint.configs.strictTypeChecked,
    // Optionally, add this for stylistic rules
    tseslint.configs.stylisticTypeChecked,
    // Other configs...
  ],
  languageOptions: {
    parserOptions: {
      project: ['./tsconfig.node.json', './tsconfig.app.json'],
      tsconfigRootDir: import.meta.dirname,
    },
    // other options...
  },
},
])
```

You can also install [eslint-plugin-react-x](https://github.com/Rel1cx/eslint-react/tree/main/packages/plugins/eslint-plugin-react-x) and [eslint-plugin-react-dom](https://github.com/Rel1cx/eslint-react/tree/main/packages/plugins/eslint-plugin-react-dom) for React-specific lint rules:

```js
```

```
// eslint.config.js
import reactX from 'eslint-plugin-react-x'
import reactDom from 'eslint-plugin-react-dom'
export default defineConfig([
  globalIgnores(['dist']),
  {
    files: ['**/*.{ts,tsx}'],
    extends: [
      // Other configs...
      // Enable lint rules for React
      reactX.configs['recommended-typescript'],
      // Enable lint rules for React DOM
      reactDom.configs.recommended,
    ],
    languageOptions: {
      parserOptions: {
        project: ['./tsconfig.node.json', './tsconfig.app.json'],
        tsconfigRootDir: import.meta.dirname,
      },
      // other options...
    },
  },
])
```

```

### *7.3 .gitignore Configurado*

El archivo .gitignore se utilizará para evitar subir archivos temporales, dependencias, configuraciones locales o información sensible al repositorio.

Configuración recomendada:

```
Logs
logs
*.log
npm-debug.log*
yarn-debug.log*
yarn-error.log*
pnpm-debug.log*
lerna-debug.log*
node_modules
dist
dist-ssr
*.local
Editor directories and files
```
```

```
.vscode/*
!.vscode/extensions.json
.idea
.DS_Store
*.suo
*.ntvs*
*.njsproj
*.sln
*.sw?
```

Esto ayuda a mantener el repositorio limpio y evitar la exposición de datos sensibles.

7.4 Configuración De Github

El repositorio de CobriXS deberá configurarse en GitHub para garantizar buenas prácticas de desarrollo y seguridad.

Branch Protection (Settings → Branches)

Configurar protección para la rama main:

- Requerir Pull Request antes de hacer merge
- Requerir al menos 1 aprobación
- Requerir que pasen los status checks de CI

- No permitir force push

Secrets (Settings → Secrets)

Se deberán configurar variables seguras para los procesos de CI/CD:

- DOCKER_USERNAME
- DOCKER_PASSWORD
- DATABASE_URL
- API_KEYS necesarias para el sistema

Github Actions (Settings → Actions)

Configurar:

- Permitir ejecución de workflows
- Habilitar GitHub Actions
- Permitir ejecución automática de pipelines CI/CD

7.5 Issues Y Project Board

Para la gestión de tareas del proyecto CobriXS, se utilizará GitHub Projects como herramienta de seguimiento del desarrollo.

Configuración:

Project Board

Nombre del tablero:

Proyecto Integrador - Sprint 1

Columnas del tablero:

- Backlog
- To Do
- In Progress
- Review
- Done

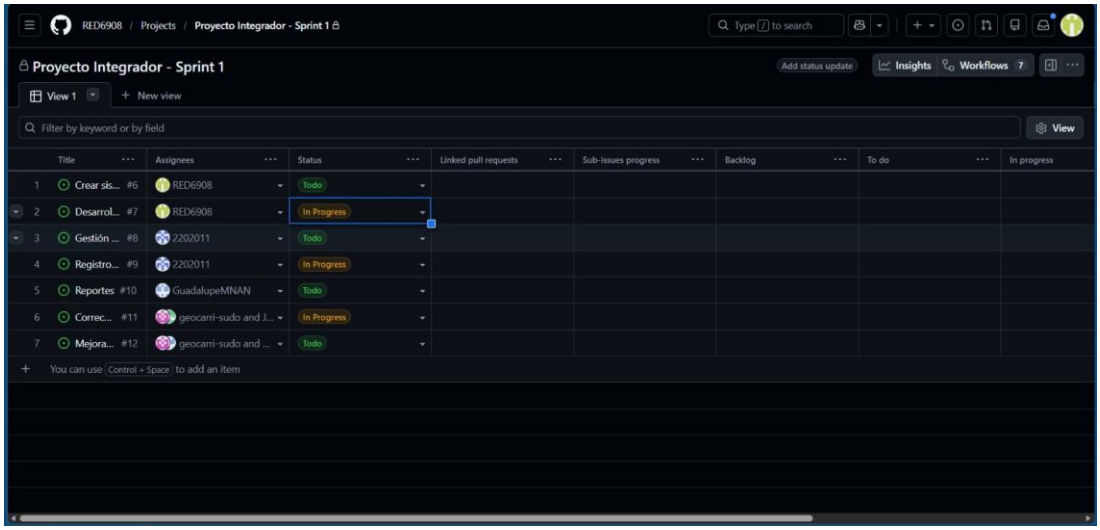


Ilustración 3. Proyecto Integrador - Sprint 1

Gestión De Issues

Se crearán issues para cada funcionalidad o tarea del sistema, incluyendo:

- Desarrollo de funcionalidades
- Corrección de errores
- Documentación
- Mejoras del sistema

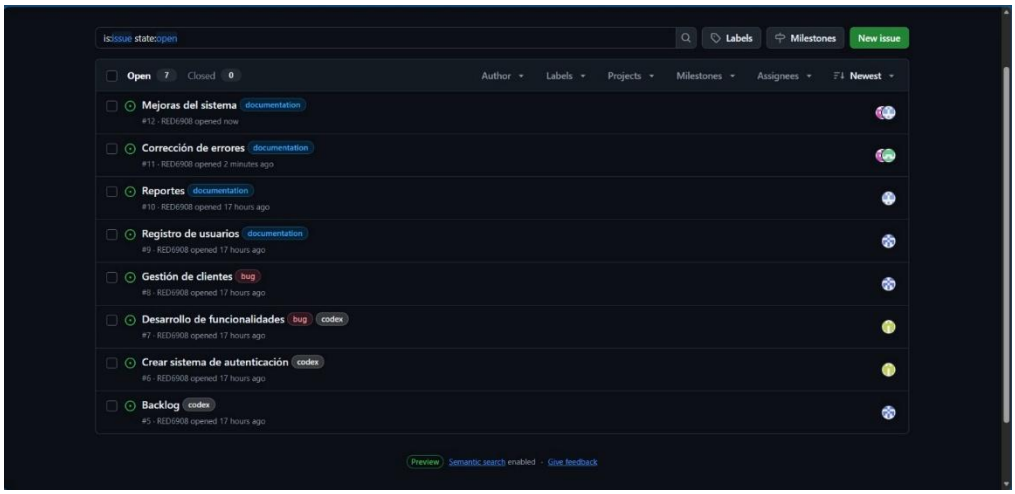


Ilustración 4. Gestión De Issues

Labels Utilizadas

- bug
- feature
- documentation
- enhancement

Cada issue será asignado a un miembro del equipo responsable de su implementación.

https://github.com/RED6908/CobriXS-v_2.git

Bibliografía

Ciclo de vida del software: todo lo que necesitas saber. (s. f.). *Intelequia*. https://intelequia.com/es/blog/post/ciclo-de-vida-del-software-todo-lo-que-necesitas-saber

Prieto, E. (2025, 2 junio). *Etapas del desarrollo de software - Tiffin University*. Tiffin University. https://global.tiffin.edu/blog/cuales-son-las-etapas-del-desarrollo-de-software

ManageEngine, communications@manageengine.com. (s. f.). *Implementación de software | ManageEngine Endpoint Central*. ManageEngine. https://www.manageengine.com/latam/desktop-central/implementacion-de-software-automatizada.html?network=g&device=c&keyword=proceso%20de%20implementaci%C3%B3n%20de%20software&campaignid=13529040230&creative=527887745417&matchtype=p&adposition=&placement=&adgroup=123087341669&targetid=kwd-880844033433&gad_source=1&gad_campaignid=13529040230&gbraid=0AAAAAChbZwVmoo1XI1dffCVjbJVCs--jw&gclid=Cj0KCQjw37nNBhDkARIsAEBGI8NUUhP5MMgvmxawm8XpC4NtFZZzAmCajyu6Rcy5FDuTG4ItxgaevD4aAkfEEALw_wcB

Conceptos básicos de HTML - Aprende desarrollo web | MDN. (s. f.). https://developer.mozilla.org/es/docs/Learn_web_development/Getting_started/Your_first_website/Creating_the_content

Amazon Web Services. (s. f.-a). *¿Qué es JavaScript (JS)?* Amazon Web Services, Inc. https://aws.amazon.com/es/what-is/javascript/

Albornoz, D. (2023, 29 junio). *Qué es React: definición, características y funcionamiento*. Tutoriales Hostinger. https://www.hostinger.com/mx/tutoriales/que-es-react

Arsys. (2025, 5 mayo). *Qué es NodeJS y ventajas que tiene este framework*. https://www.arsys.es/blog/nodejs-framewok

Supabase: Qué es, cómo funciona y por qué usarlo en 2026. (s. f.). https://www.nocodehackers.es/herramientas-no-code/supabase

Worry-Free managed PostgreSQL Hosting | DigitalOcean. (s. f.). https://www.digitalocean.com/products/managed-databases-postgresql?utm_source=google&utm_medium=cpc&utm_campaign=search_nb_databases_postgres_converted_la_en&utm_adgroup=&utm_term=postgres%20cloud&utm_creative=770944719309&utm_location=9136671&utm_matchtype=e&utm_device=c&gad_source=1&gad_campaignid=22933189427&gbraid=0AAAAADw9jcu8yMOgwpGa8cNftotD3400l&gclid=Cj0KCQjw37nNBhDkARIsAEBGI8Mn7crp8ypUJ-oQztqFGJdO8oAv_xa9iTzWwL9XKl5Ry72hiDweAE8aAqN1EALw_wcB

McCue, I. (2021, 26 mayo). *ERP Modules: Types, Features & Functions*. Oracle NetSuite. https://www.netsuite.co.uk/portal/uk/resource/articles/erp/erp-modules.shtml

Patrones de modularización comunes. (s. f.). Android Developers. https://developer.android.com/topic/modularization/patterns?hl=es-419

Tipos de usuarios, roles y privilegios—Portal for ArcGIS | Documentación de ArcGIS Enterprise. (s. f.). https://enterprise.arcgis.com/es/portal/11.2/administer/windows/roles.htm

Introducción a las funciones de usuario - Soporte técnico de Microsoft. (s. f.). https://support.microsoft.com/es-es/office/introducci%C3%B3n-a-las-funciones-de-usuario-9c77bb88-cb2e-4fad-829b-5bd4c807b757

Control de acceso basado en roles (descripción general) - Guía de administración del sistema: servicios de seguridad. (2011, 1 enero). https://docs.oracle.com/cd/E24842_01/html/E23286/rbac-1.html

GitHub Careers. (s. f.). GitHub Careers. https://www.github.careers/careers-home

Sistemas_A6vftt6a. (2022, 22 noviembre). *Entornos de desarrollo: descripción y tipologías*. Arroba Solutions. https://www.arrobasolutions.com/entornos-desarrollo-descripcion-tipologias/

Config Tool Introduction and Use. (2024, 1 noviembre). NXP Community. https://community.nxp.com/t5/i-MX-Processors-Knowledge-Base/Config-Tool-Introduction-and-Use/ta-p/1986035

Variables de Entorno: ¿qué es y cómo funcionan? | Lenovo México. (s. f.). https://www.lenovo.com/mx/es/glosario/variables-de-entorno/?orgRef=https%253A%252F%252Fwww.google.com%252F&srsltid=AfmBOoq2m62tsTP7BSkLiQ75C9re1OUXzRKkm4gTCpN-SMscfiJJJ3QS

Daityari, S. (2025, 17 febrero). *Las 12 mejores herramientas de revisión de código para desarrolladores (edición 2026)*. Kinsta®. https://kinsta.com/es/blog/herramientas-de-revision-de-codigo/

Atlassian. (s. f.-b). *Infraestructura como código (IaC) | Atlassian*. https://www.atlassian.com/es/microservices/cloud-computing/infrastructure-as-code

Son, H. (2026, 2 marzo). How To Create A Test Plan: Steps, Examples, & Template - TestRail. *TestRail | The Quality OS for QA Teams*. https://www-testrail-

com.translate.goog/blog/create-a-test-plan/?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc

Caballero, Y. P. (2024, 9 julio). *¿Qué son las pruebas unitarias y cómo llevar una a cabo?* Yeeply. https://yeeply.com/blog/tendencias-habilidades/que-son-pruebas-unitarias/

Viele, M. (2025, 2 enero). *¿Qué son las pruebas de integración en el desarrollo de software? Digital.ai*. Digital.ai. https://digital.ai/es/glossary/what-is-integration-testing/

Pruebas End-To-End: definición, ejemplos y herramientas. (s. f.). https://qalified.com/es/blog/end-to-end-testing/

Odoo S.A. (s. f.). *Ambiente de pruebas*. Odoo. https://www.odoo.com/es/slides/slide/ambiente-de-pruebas-10575

Testim. (2025, 7 marzo). Test Case Design: a Guide for QA Engineers With Examples. *AI-driven E2E automation with code-like flexibility for your most resilient tests*. https://www.testim.io/blog/test-case-design-guide-for-qa-engineers/

GitLab. (s. f.). *¿Qué es el control de versiones?* about.gitlab.com. https://about.gitlab.com/es/topics/version-control/

Git - ¿Qué es una rama? (s. f.). https://git-scm.com/book/es/v2/Ramificaciones-en-Git-%C2%BFQu%C3%A9-es-una-rama%3F

GitKraken. (2023, 8 agosto). What is a Pull Request in Git? | Intermediate Git Tutorial. *GitKraken*. https://www.gitkraken.com/learn/git/tutorials/what-is-a-pull-request-in-git

Understand deployment strategies. (2022, 10 mayo). Oracle Help Center. https://docs.oracle.com/es/solutions/plan-mad-strats-devops/understand-depoyment-architectures.html

IBM i. (s. f.). https://www.ibm.com/docs/es/i/7.5.0?topic=statements-rollback

Datadog. (2025, 22 julio). *What are Metrics & What are the Most Important Metrics to Monitor?* Datadog. https://www.datadoghq.com/knowledge-center/metrics/

Issue board · Project · User · Help. (s. f.). GitLab. https://visorus.com.mx/git/help/user/project/issue_board.md